SIMATS ENGINEERING
ASSESSMENT TOOL 1
COURSE CODE: CSA14
COURSE NAME: COMPILER DESIGN
COURSE OUTCOME COVERED
CO5: Develop code optimization and Code Generation
using DAG representation
with data flow analysis to produce optimized target code. (BL3)
Assessment Tool Weight-age: 40%
QUESTIONS: 5
Total Marks:50
Annexure A
SIMULATION TASK
Code of Conduct:
I V.Santhosh Kumar /192411015(Name / Reg No) certify that this submission is my original work
and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.
Signature of the student: V.Santhosh Kumar

Criteria	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Weightage (%)
Conceptual Understanding	Demonstrates strong understanding of underlying concepts in the simulation	Shows good understanding with minor gaps	Partial understanding; some misconceptions	Lacks understanding of key concepts	20%

Model Design 25%	Develops accurate, well-structured simulation/model independently	Develops correct model with minor errors	Model is incomplete or requires guidance	Unable to develop a proper model
Tool Usage 20%	Uses simulation tools effectively with correct setup and execution	Uses tools with minor errors or guidance	Limited ability to use tools properly	Unable to use simulation tools
Analysis of Results 20%	Provides clear,			

Annexure A

SIMULATION TASK

QUESTIONS

1.Given the three-address code sequence:

```
t1 = x * 1
t2 = t1 + y
t3 = t2 - 0
t4 = t3 * 0
if t4 != 0 goto L1
t5 = y + y
```

Simulate the peephole optimization process on this sequence. Produce the optimized instruction sequence and justify each optimization step applied, including algebraic simplification, constant folding, and dead code elimination.

2.Partition the following intermediate code into basic blocks and construct the corresponding
control flow graph. Identify all leader statements and mention the rules used to identify them:

```
(1) x = a + b
(2) y = x * c
(3) if y > 50 goto 6
(4) z = y - d
(5) goto 7
(6) z = y + d
(7) w = z * 2
```

3. Construct the DAG for the following basic block:

```
p = a + b
q = a + b
r = p * q
s = a + b
t = r + s
```

Using the DAG, identify all common sub-expressions. Then simulate a simple code generator
to produce optimized target code and explain the register allocation decisions at each step.

4. For a target machine with two registers R0 and R1, simulate the working of a simple code
generator for the code:

```
t1 = a + b
t2 = c * d
t3 = t1 - t2
t4 = e + f
t5 = t3 / t4
```

Show the register descriptor and address descriptor after each instruction is processed.

5. For a RISC-style target machine with instructions LOAD, STORE, ADD, SUB, MUL, MOV, simulate target code generation for the expression:

```
(a - b) * (c + d) - e
```

Show the complete sequence of target instructions generated. Also explain the runtime storage management and stack frame layout used during the computation.

Given the three-address code sequence:
t1 = a + b,
t2 = t1 + 0,
t3 = t2 * 1,
if t3 == 0 goto L1,
t4 = t3 + t3

Answers

Question 1 — Peephole Optimization

Given three-address code:

```
t1 = x * 1
t2 = t1 + y
t3 = t2 - 0
t4 = t3 * 0
if t4 != 0 goto L1
t5 = y + y
```

Step-by-step Peephole Optimization

Step	Original	Optimization Applied	Result
1	t1 = x * 1	Algebraic simplification (multiplying by 1 is redundant)	t1 = x
2	t2 = t1 + y	Copy propagation (t1 = x, substitute)	t2 = x + y
3	t3 = t2 - 0	Algebraic simplification (subtracting 0 is redundant)	t3 = t2 => t3 = x + y
4	t4 = t3 * 0	Constant folding / algebraic simplification (anything * 0 = 0)	t4 = 0
5	if t4 != 0 goto L1	Constant folding on the condition: t4 is now the constant 0, so "0 != 0" is always FALSE	Branch never taken -> statement removed
6	t5 = y + y	No simplification applies (y is not a constant); retained as-is	t5 = y + y

Dead Code Elimination

Once the conditional branch is folded to "always false", the goto L1 statement is dead code and is removed. This means t4 is no longer used anywhere, so the instruction that computes it

(t4 = t3 * 0) is also dead. By the same reasoning t3 (used only to compute t4) and, in turn, t2
(used only to compute t3) become dead once their sole consumer disappears — unless they
are required as live-out values of the block. Assuming t1–t4 are not needed beyond this block,
the entire chain collapses:

```
t5 = y + y
```

Control Flow Graph

Figure 1: Control Flow Graph — B1 branches to B3 when y > 50 (true) and falls through to B2 otherwise; both B2 and B3 converge into B4.

Edge summary: B1 →B3 (true branch of the condition), B1 →B2 (fall-through / false branch), B2 →B4 (via the unconditional goto 7), B3 →B4 (fall-through into the next leader).

Question 3 — DAG Construction & Code Generation

Given basic block:

```
p = a + b
q = a + b
r = p * q
s = a + b
t = r + s
```

DAG Construction

While building the DAG, before creating a new node for each operation we check whether an identical node (same operator, same operands) already exists. The expression a + b is computed three times (for p, q and s); since a and b are unchanged throughout the block, all three assignments refer to the exact same node, so only one "+" node is created and labelled with all three names.

Figure 2: DAG for the basic block — the shared node labelled p, q, s shows the common sub-
expression a + b.

Common Sub-expressions Identified

- a + b is computed once and reused for p, q and s — this is the only common sub-expression in the block.
- p * q reduces to (a+b) * (a+b), i.e. the same node multiplied by itself, to compute r.
- r + s reduces to r + (a+b) using the same shared node again, to compute t.

Optimized Target Code (from DAG traversal)

```
p = a + b
q = p
; q is the same value as p (common sub-expression)
r = p * p
; r = p * q, but q = p
s = p
; s is the same value as p
t = r + p
; t = r + s, but s = p
```

Simple Code Generator Simulation (Registers R0, R1)

Instruction
Generated
Action
Register
Descriptor
Address
Descriptor

Instruction Generated	Action	Register Descriptor	Address Descriptor
MOV a, R0	Load a into a free register R0	R0 = {a}	a: {R0, memory}
ADD b, R0	R0 = R0 + b, computing p (=q=s)	R0 = {p, q, s}	p,q,s: {R0}
MOV R0, R1	Copy p into R1 to preserve it before R0 is reused for r	R0={p,q,s}, R1={p,q,s}	p,q,s: {R0,R1}
MUL R1, R0	R0 = R0 * R1, computing r = p * p; p no longer needed in R0	R0 = {r}, R1 = {p,q,s}	r: {R0}; p,q,s: {R1}
ADD R1, R0	R0 = R0 + R1, computing t = r + p	R0 = {t}, R1 = {p,q,s}	t: {R0}; p,q,s: {R1}
MOV R0, t	Store t back to memory (block exit, t is live-out)	R0 = {t}	t: {R0, memory}

Register allocation decisions: R0 is used first because both registers start empty (getReg picks any free register). A copy of p is kept in R1 before r is computed, because p is still needed later for t = r + p and R0 must be reused as the destination for the multiplication. Once p is copied to R1, R0 is free to be overwritten by r, and once r is consumed to form t, R0 is safely reused again. This reuse of only two registers is possible precisely because the DAG exposed

that q and s are copies of p rather than independent computations — without common sub-
expression elimination, three separate additions and more register pressure would have been
required.

Question 4 — Simple Code Generator (2 Registers R0, R1)

Given three-address code:

t1 = a + b
t2 = c * d
t3 = t1 - t2
t4 = e + f

```
DIV  R1,  R0
; R0 = t5 = t3 / t4 (R1 freed, t4 is dead)
MOV  R0,  t5
```

memory instead of kept in registers
Bottom (top of
stack)
Result / return value
slot
Location where the final computed value is

RUBRICS FOR CALCULATION

Simulation tools	Criteria Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Conceptual Understanding	20%	Demonstrates strong understanding of underlying concepts in the simulation	Shows good understanding with minor gaps	Partial understanding; some misconceptions	Lacks understanding of key concepts
Model Design	25%	Develops accurate, well-structured simulation/model independently	Develops correct model with minor errors	Model is incomplete or requires guidance	Unable to develop a proper model
Tool Usage	20%				

Uses simulation
tools effectively
with correct setup